University of Guelph
College of Engineering
Department of Electrical and Computer Engineering

Tic-Tac-Toe Project Report

Mini-Project 1
Group 63

November 8th, 2026

ENGG*1410
Bogdan Bunescu

Authors:
William Bailey | 1387068
Myles Leclerc | 1391784

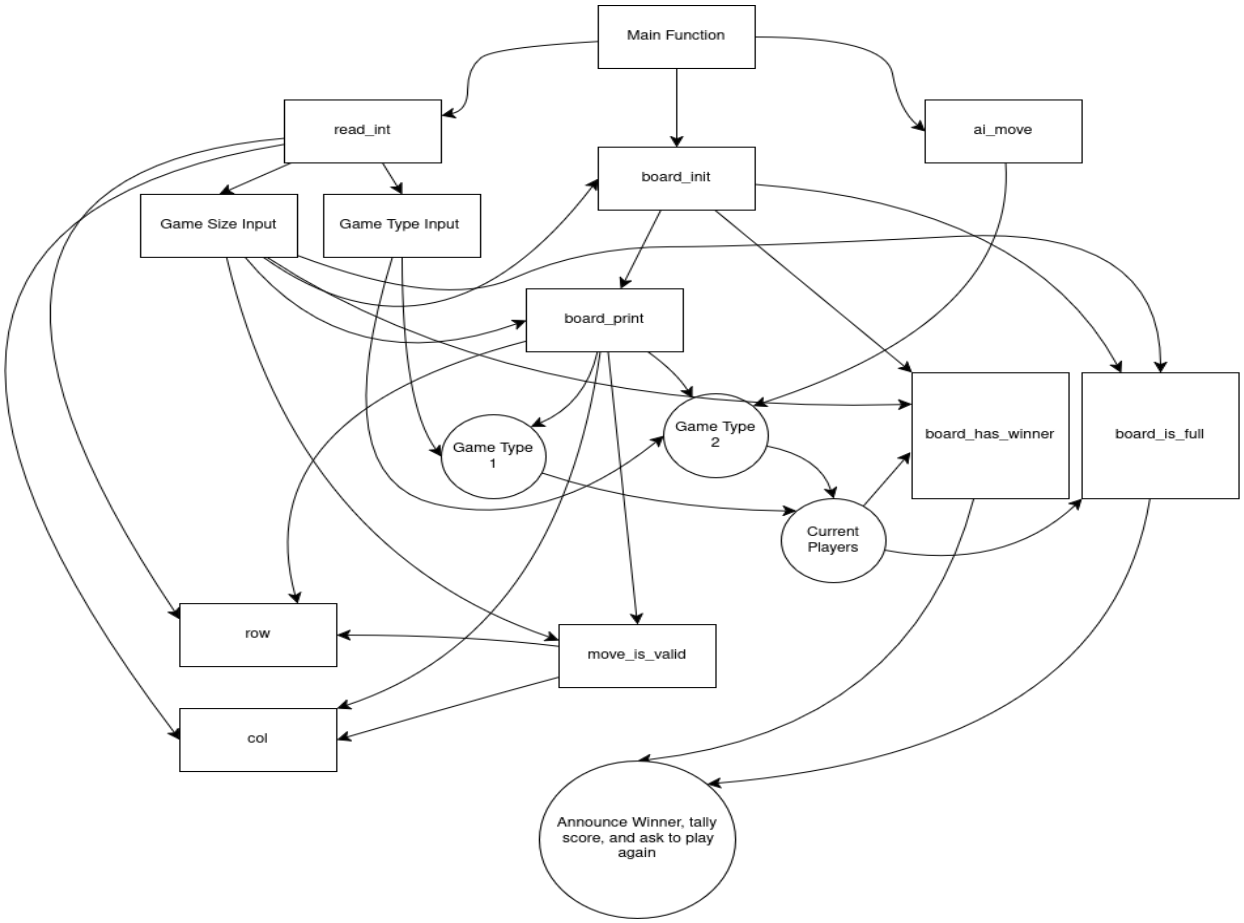
Problem Statement

The Tic-Tac-Toe program operates using three .c code files and two .h header files. The provided input.c and input.h files validate user input whenever the main file requires data from the player, ensuring it passes standard checks before being used.

The board.c file and its corresponding header describe how the game board operates. The six functions we wrote handle initializing and printing the board, filling spaces with player or AI inputs, and checking for winning patterns. The main function calls these whenever a board operation is needed.

Finally, the main file contains the actual game logic. It asks the player to select a board size and choose between two-player or AI gameplay. The board is then initialized and displayed, and players take turns entering moves. The game continues until someone wins or no moves remain. The program then announces the result, updates the tally score, and asks whether the player wants to play again or exit.

Design

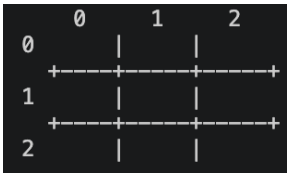
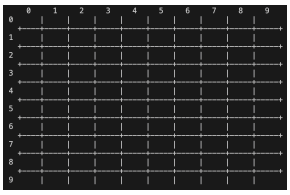
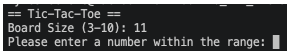
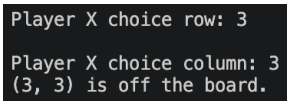


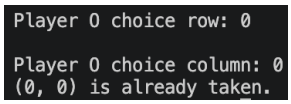
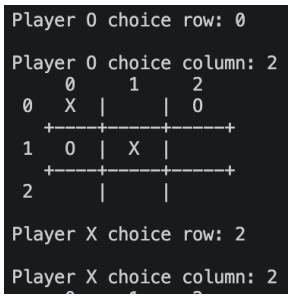
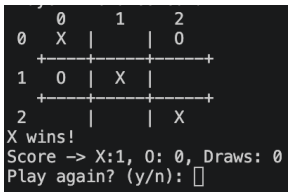
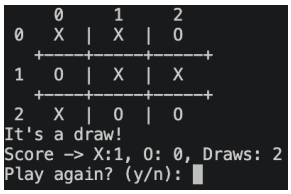
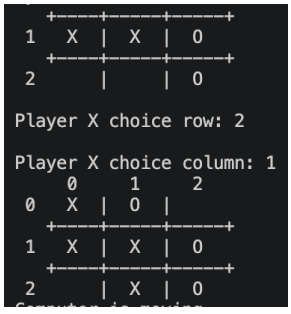
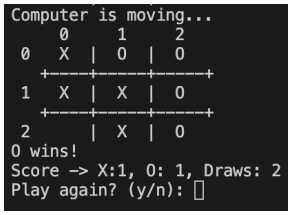
Pseudocode for ai_move:

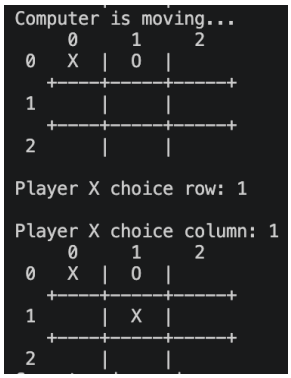
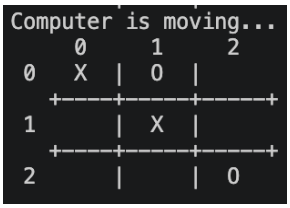
1. Attempt to win if possible
    a. Read the board for empty spot
    b. If an empty spot is found mark it
        i. If marking that spot wins, return
    c. Otherwise put EMPTY back
2. Block any winning cells of the opponent
    a. Read the board for empty spot
    b. The board is marked by a human
        i. If the board has a winner by a human mark, mark it instead and return
    c. Otherwise put EMPTY back
3. Move on the first cell that isn't empty
    a. If the board is empty mark it.

Testing

Note that when viewing this table, these are the results of the finished (i.e. what we submitted) and not any test results throughout our commits, as main.c was the last file that was finished.

Scenario	Input	Expected Result	Actual Result
Size 3	Board Size (3-10): 3	A 3x3 grid	 (Same as expected result, just numbered starting at 0, not 1)
Size 10	Board Size (3-10): 10	A 10x10 grid	 (Same as expected)
Invalid size	Board Size (3-10): 11	Program asks for "a number within the range". With no errors or warnings.	 (Same as expected)
An off-board move	Player X choice row: 3 Player X choice column: 3	Prints that (3, 3) is out of bounds since the reference coordinate is (0,0) and not (1,1). With no	

		errors or warnings.	
An occupied cell	Player X choice row: 0 Player X choice column: 0 //Displays board// Player O choice row: 0 Player O choice column: 0	Printing out that the cell is already taken. With no errors or warnings.	
A diagonal win		Terminal printing out the table, “X wins!” and then printing out the running totals for X wins, O wins, and draws. Asking to play again at the end. With no errors or warnings.	
A draw	//Exchanges between player and opponent// Player X choice row: 1 Player X choice column: 2	Terminal would print the table, “It’s a draw!” and then print of the running totals for X wins, O wins, and draws. Asking to play again at the end. With no errors or warnings.	
The computer taking a win	//Exchanges between player and computer/AI// 	Terminal prints the board with 3 ‘O’ s in column 2. Then prints “O wins!” followed by printing the running tally for X wins, O wins, and draws. Asking to play again at the end. With no errors or warnings.	

The computer blocking		The terminal would print “Computer is moving…”then print the table with an ‘O’ at (2, 2) to block the possible win.	

Error Analysis

Bug Encountered	How we noticed / located	Fix
Code did not stop when ‘n’ was selected to play again.	The code that reprinted game size and game mode at lines 108-113 had no conditional statement to check if the input was ‘n’.	Added a conditional break statement to check if the user wanted to play again. “if (redo=='n'){break;}”
Tic-Tac-Toe Board printed with no new line. Despite one being added. This left the board not being properly spaced out from the text.	When reading over the board print function; below line 43 a remnant of code imported from lab3part1 was found as Char cell = ‘ ‘	Deleting the line removed the moved the error. The reason why it was causing the error was because it was being redundantly checked on line 46.
Code not being compiled due to an include error in the main function.	#include board.c & input.c were redundant in the include and the compiler threw out an error.	Removed the .c includes and only kept the header files.